

Topic 1: Inheritance, Interfaces and Design patterns

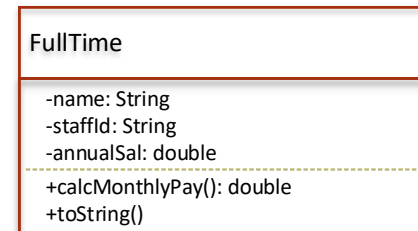
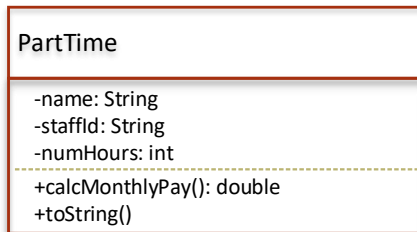
Part 1: Inheritance

Agenda

1. Introduction
2. Inheritance
3. Method overload
4. Method override
5. Polymorphism
6. Abstract classes
7. Exercise
8. Multiple inheritance
9. Summary
10. Questions and comments

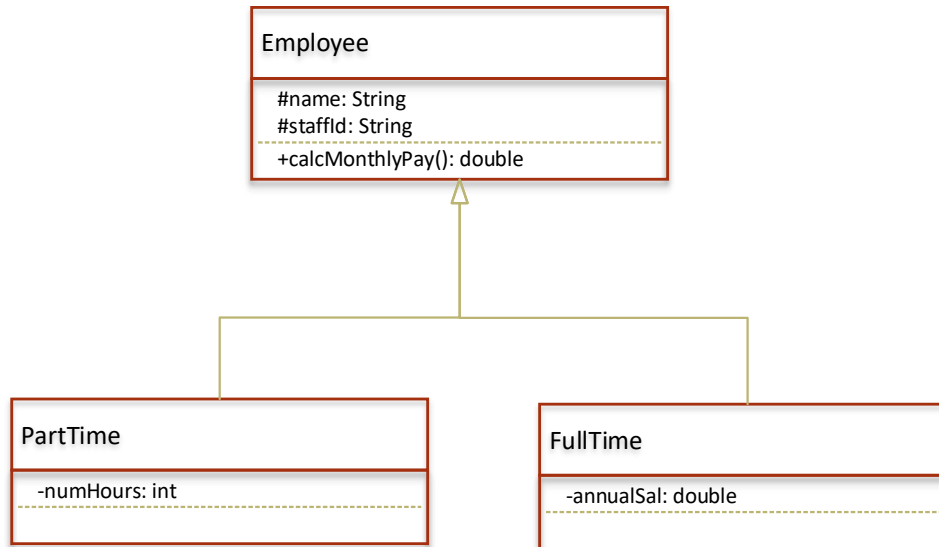
1. Introduction

- Classes for a program to calculate monthly pay:
 1. Part time: $\text{numHours} * \text{rate}$
 2. Full time: $\text{annual salary} / 12$
 - What is wrong? Do you see duplication? How can you fix?



1. Introduction

- **Refactor** by creating a superclass to remove duplication (note the **access levels of members in superclass**)
 1. Can you write the method `calcMonthlyPay()` in `Employee`?
 2. What is a concrete class?
 3. What is an abstract class?



Introduction

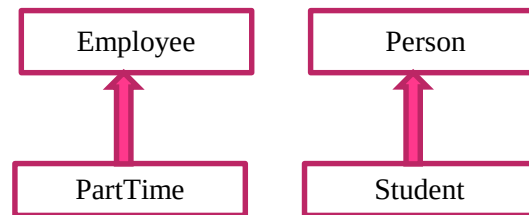
- **Review of OOP** covered OOP concepts
 1. Objects, classes, encapsulation and abstraction
 2. How objects (and classes) interact
 3. Benefits of encapsulation and abstraction
 4. Why it is important to separate POJO and Tester
 5. How to write POJO and a Tester
- There are several structures in OOP (e.g., classes, interfaces, etc.)
- There are several interactions (relationships) in OOP

Introduction

- Relationships show how classes work together (interact) to solve a problem
- Three common relationships in OOP
 1. Dependence: one class uses (depends on) another; e.g.:
 - Tester uses Sum (see program in Overview of OOP)
 2. Inheritance
 - a. A class inherits another class
 - b. **is a** relationship
 - c. Example: Student **is a** Person
 3. Composition/Aggregation
 - a. A class consists of other class(es) / is part of another class
 - b. **has a / is part of** relationship
 - c. Example:
 - Lecturer **has** Student
 - House **has** Room (or Room **is part of** House)
 - d. **NOTE: this concept will be used later in the course**

2. Inheritance

- Allows us to create classes from an existing classes
 1. Subclasses from existing class
 2. Super class from existing classes
- A subclass **inherits the members** (i.e., data and methods) of a superclass

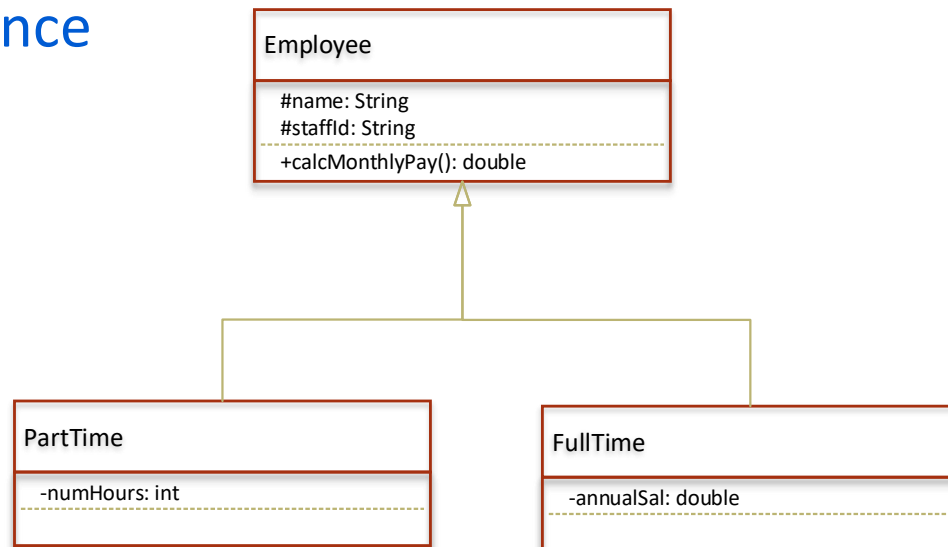


- A subclass can also have additional members
- T/F: PartTime **is an** Employee? Employee **is** PartTime?
- T/F Student is a Person; Person **is a** Student?
- A subclass can **override** methods from superclass

2. Inheritance

■ Benefits

1. Code re-use, e.g., **Employee** is re-used (written once, parts used many times in **PartTime** and **FullTime**)
2. Code maintenance, e.g., (changes: adding new members) to class one class (e.g., **PartTime**) may not affect others
3. Reduces development time and cost
4. Using the example below, explain some of the benefits of inheritance



2. Inheritance (how to program)

■ Create objects

1. Group the members (data and methods): **Encapsulation**
2. Determine access levels of members: **Abstraction**

Symbol	Access restriction	Java keyword
-	Private (accessible at class level	private
#	Protected (accessible at class and subclass level	protected
~	Accessible at package level)	
+	Accessible by all classes	public

■ Create UML class diagram

1. Create the superclass (**generally data protected; methods public**)
2. There are different types of super classes: **concrete and abstract**
3. Create subclasses (**generally data private; methods public**)
4. **NOTE: Java does not allow multiple inheritance** (so resolve multiple inheritance)

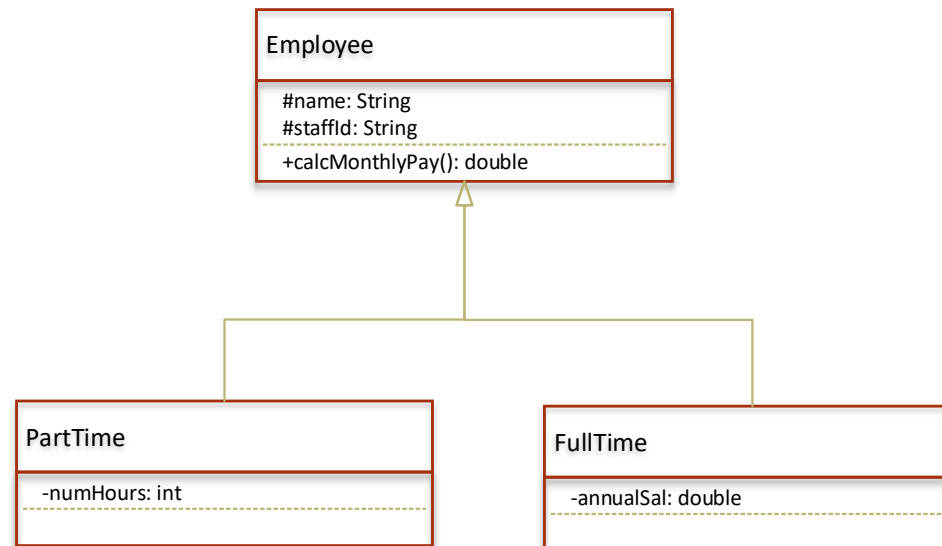
■ Convert class diagrams to Java programs

■ Create **Tester** that **uses** the super and sub classes

2. Inheritance: Example

- The UML diagram below shows the classes **Employee**, **FullTime** and **PartTime**. Convert to a Java program. Calculate monthly pay is as:

1. Part time: numHours * rate
2. Full time: annual salary /12
3. NOTE: all the classes are **concrete**



2. Inheritance: Example

■ Superclass: Employee

```
package ex1;
public class Employee {
    protected String name;
    protected String staffid;

    public Employee() {
        name = "None";
        staffid = "000000";
    }
    public Employee(String namePar, String staffidPar) {
        name = namePar;
        staffid = staffidPar;
    }
    ...
    ...
    public double calcMonthlyPay() {
        return 0;
    }
    public String toString() {
        return "StaffId: " + staffid + "\nName: " + name +
            "Monthly pay: R"+calcMonthlyPay();
    }
}
```

Not good programming

- Employee
 - ◆ name : String
 - ◆ staffid : String
 - Employee()
 - Employee(String, String)
 - getName() : String
 - setName(String) : void
 - getStaffid() : String
 - setStaffid(String) : void
 - calcMonthlyPay() : double
 - toString() : String

2. Inheritance: Example

- Subclass **PartTime** extends superclass **Employee**
 - Can you observe code re-use and benefits?

```
package ex1;
public class PartTime extends Employee {
    private int numHours;

    public PartTime() {
        super();
        numHours = 0;
    }
    public PartTime(String namePar, String staffidPar, int numHoursPar) {
        super(namePar, staffidPar);
        numHours = numHoursPar;
    }
    ...
    ...
    public double calcMonthlyPay() {
        return numHours * 80;
    }
    public String toString() {
        return super.toString() +
            "\nHours:\t" + numHours + "\tRate: R" + 80;
    }
}
```

PartTime inherits and overrides method

PartTime inherits and overrides method

PartTime

- ▣ numHours : int
- ^c PartTime()
- ^c PartTime(String, String, int)
- getNumHours() : int
- setNumHours(int) : void
- [▲] calcMonthlyPay() : double
- [▲] toString() : String

2. Inheritance

- Tester creates instances of the classes **Employee**, **PartTime**

Tester
main(String[]) : void

```
package ex1;
public class Tester {
    public static void main(String[] args){
        PartTime p = new PartTime("N Langa", "123456", 10);
        System.out.println(p);
        Employee e = new Employee ();
        e = p;
        System.out.println(e);
    }
}
```

You can create instances of concrete classes

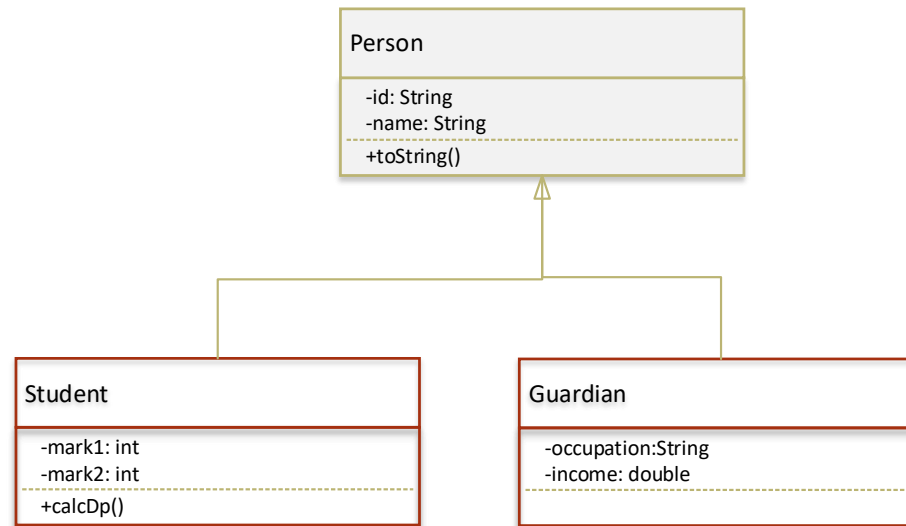
- Output

```
StaffId: 123456
Name: N Langa
Monthly pay: R800.0
Hours: 10    Rate: R80
StaffId: 123456
Name: N Langa
Monthly pay: R800.0
Hours: 10    Rate: R80
```

You can assign an instance of a subclass to a variable of a superclass

2. Inheritance: Exercise 1

- See Topic 1 Exercise 1: Inheritance (based on the diagram below)
- Complete classes Person and Student only and a tester



2. Inheritance: Important concepts

■ What is a:

1. **Method header**? Return type and parameter list (access level is optional), e.g.:
double calcMonthlyPay()
2. **Method signature**? Method name and parameter list, e.g.:
calcMonthlyPay()
3. **Method declaration**? Create from the header (access level optional), e.g.:
4. **public double calcMonthlyPay();**
5. **Method implementation**? How the method works, e.g.,

```
public double calcMonthlyPay() {  
    return 0;  
}
```

■ Important concepts

1. Method overload
2. Method override
3. Polymorphism

3. Method overload

- Method overload: when the same method exists in many forms)
 - Same method name but **different parameter list** (i.e., **different signatures**)
 - Occurs in the same class

```
package ex1;
public class Employee {
    protected String name;
    protected String staffid;

    public Employee() {
        name = "None";
        staffid = "000000";
    }
    public Employee(String namePar, String staffidPar) {
        name = namePar;
        staffid = staffidPar;
    }
    ...
    ...
    public double calcMonthlyPay() {
        return 0;
    }
    public String toString() {
        return "StaffId: " + staffid + "\nName: " + name +
            "Monthly pay: R"+calcMonthlyPay();
    }
}
```

Overload

Question: can you identify the overloaded methods in PartTime?

Not good programming

3. Method overload

- What is wrong with the code below

```
public class Student extends Person {  
    .....  
    public int calcDp() {  
        int dp = (int)(Math.round(mark1+mark2)/2);  
        return dp;  
    }  
    public double calcDp() {  
        int dp = (int)(Math.round(mark1+mark2)/2);  
        return dp;  
    }  
    .....  
}
```

- Answer?

4. Method override

- Method override: same method exists in many forms **in superclass and subclass**
 - **Subclass** inherits from a **superclass** and implements it differently

```
package ex1;  
public class PartTime extends Employee {  
    private int numHours;
```

```
    public PartTime() {  
        super();  
        numHours = 0;
```

```
    }  
    public PartTime(String namePar, String staffidPar, int numHoursPar) {  
        super(namePar, staffidPar);  
        numHours = numHoursPar;
```

```
    }  
    ...  
    ...
```

```
    public double calcMonthlyPay() {  
        return numHours* 80;
```

```
    }  
    public String toString() {  
        return super.toString()+  
            "\nHours:\t"+ numHours +"\tRate: R" + 80;
```

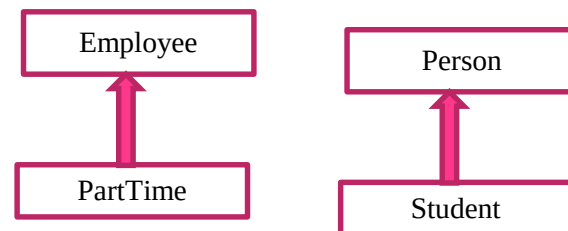
```
    }  
}
```

1. Which methods are overloaded?
2. Which other method is overridden?

PartTime inherits and overrides method

5. Polymorphism

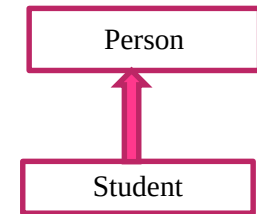
- Polymorphism: many forms (multiple existence)
- Polymorphism relates to **overload** and **override**
- **Overload**
 1. Different behaviour according to the method selected in the class
 2. Method selection is at compile time (static binding)
- **Override**
 1. Different **behaviour** by subclass, e.g.,
 - a. **PartTime** can behave as **Employee**
 - b. **Student** can behave as **Person**



2. **Method** selection is at run time (i.e., dynamic binding)

5. Polymorphism

- Student **is a** Person
 - Student object can exist in 2 forms
 - Depending on the context, a Student **object** can **behave** as Student or Person
- Person **is a** Student (**wrong**)
- How is polymorphism achieved?
 - Question: using the **Person-Student** relationship, explain the term **Polymorphism** in your own?



5. Polymorphism

- Polymorphism and dynamic binding: example
 1. Create an instance each of **Employee** and **PartTime**
 2. Which method will be called to display in the code below? Explain your answer.
- Answer: a call to toString() in **Employee**? **Student?**

```
package ex1;
public class Tester {
    public static void main(String[] args){
        PartTime p = new PartTime("N Langa", "123456", 10);
        System.out.println(p);
        Employee e = new Employee ();
        e=p;
        System.out.println(e);
    }
}
```

```
StaffId: 123456
Name: N Langa
Monthly pay: R800.0
Hours: 10    Rate: R80
StaffId: 123456
Name: N Langa
Monthly pay: R800.0
Hours: 10    Rate: R80
```

- Output

6. Abstract class

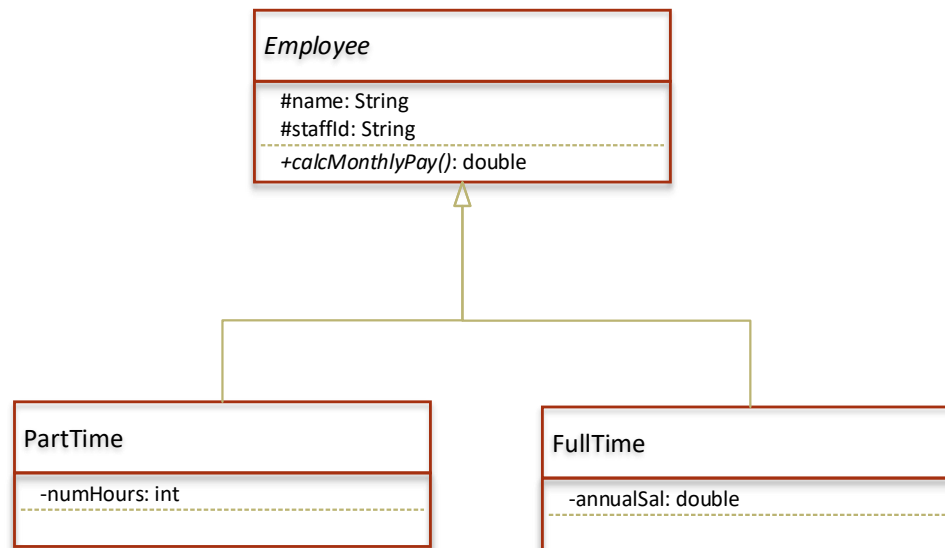
- A concrete class: a class that can be instantiated (i.e., can directly create an instance of the class)
- Abstract class: a class that cannot be instantiated (i.e., cannot directly create an instance)
- Any real-life example?
 1. Vehicle? **Vehicle** is abstract; **Car, Bike, etc.** are more concrete classes
 2. **Person** is abstract; **Student, Employee, Client, etc.** more concrete
- An abstract class must be stable because **changes affect the subclasses**
- When do you create an abstract class?
 - It depends on the needs of your program
 - **Will you need to create objects?** If your answer is
 - **Yes:** you need concrete class
 - **No:** you need an abstract class
 - But **if the class contains an abstract** method then the class **MUST be declared abstract**

6. Abstract class

- The main purpose of an abstract class is inheritance and code re-use (by subclasses)
- In UML diagram:
 1. *Italic* class name => abstract class
 2. *Italic* method name => abstract method
- An abstract class is like your normal class. The main difference is the keyword **abstract** (to show a class is abstract)
- Any class **MAY** be declared abstract
- **BUT any class that contains an abstract method MUST be declared abstract**
- You cannot create instances of an abstract class
 1. **But you can declare a variable of an abstract class**
 2. An abstract class can be inherited by another class (concrete or abstract)

6. Abstract class

- Any class **MAY** be declare abstract
 - Example: **Employee-PartTime-FullTime**
 - Employee as abstract (see the class name is **italic**)



6. Abstract class

- ❑ Any class **MAY** be declared abstract (but remember you cannot create instances of that class)
- ❑ NOTE: **PartTime** class **DOES NOT** change

```
package ex2;
public abstract class Employee {
    protected String name;
    protected String staffid;

    public Employee() {
        name = "None";
        staffid = "000000";
    }

    public Employee(String namePar, String staffidPar) {
        name = namePar;
        staffid = staffidPar;
    }
    ...
    public double calcMonthlyPay() {
        return 0;
    }
    public String toString() {
        return "StaffId: " + staffid + "\nName: " + name +
            "Monthly pay: R"+calcMonthlyPay();
    }
}
```

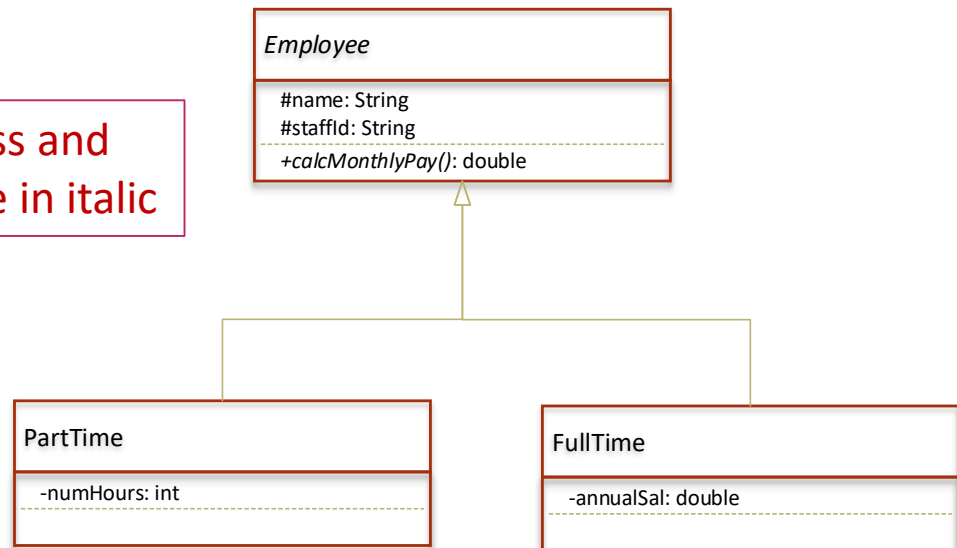
```
package ex2;
public class Tester {
    public static void main(String[] args){
        PartTime p = new PartTime("N Langa", "123456",10);
        System.out.println(p);
        //Employee e = new Employee();//cannot create an instance of Employee
        Employee e1=p;//can assign an instance of PartTime to variable of Employee
        System.out.println(e1);
    }
}
```

Cannot create instance of Employee because it is an abstract class

6. Abstract class

- A class which contains an abstract method must be declared abstract
 - Employee is abstract because calcMonthlyPay() is abstract
 - Question: why must calcMonthlyPay() be **an abstract method**?

NOTE: in UML, abstract class and abstract method names are in italic



6. Abstract class

- A class which contains an abstract method must be declared abstract

```
package ex2;
public abstract class Employee {
    protected String name;
    protected String staffid;

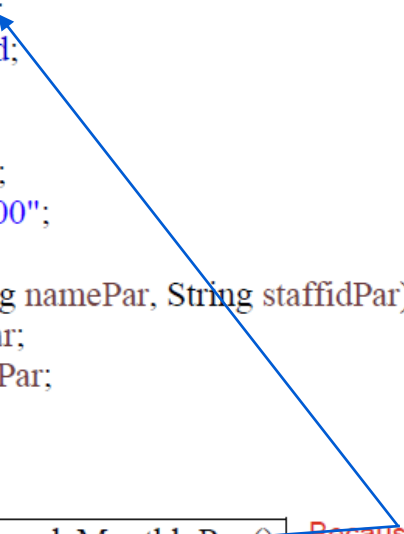
    public Employee() {
        name = "None";
        staffid = "000000";
    }

    public Employee(String namePar, String staffidPar) {
        name = namePar;
        staffid = staffidPar;
    }

    ...
    ...

    public abstract double calcMonthlyPay();

    public String toString() {
        return "Staffid: " + staffid + "\nName: " + name +
            "Monthly pay: R"+calcMonthlyPay();
    }
}
```



Because this method is abstract,
the class MUST be abstract

- NOTE: PartTime class DOES NOT change

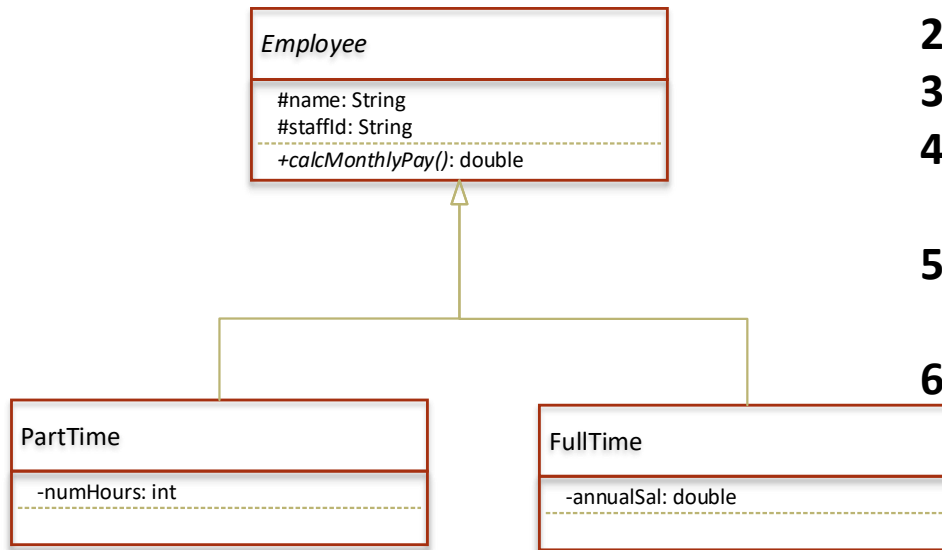
6. Abstract class

- A class which contains an abstract method **must** be declared abstract

```
package ex2;
public class Tester {
    public static void main(String[] args){
        PartTime p = new PartTime("N Langa", "123456",10);
        System.out.println(p);
        //Employee e = new Employee();//cannot create an instance of Employee
        Employee e1=p;//can assign an instance of PartTime to variable of Employee
        System.out.println(e1);
    }
}
```

Cannot create instance of Employee
because it is an abstract class

7. Exercise

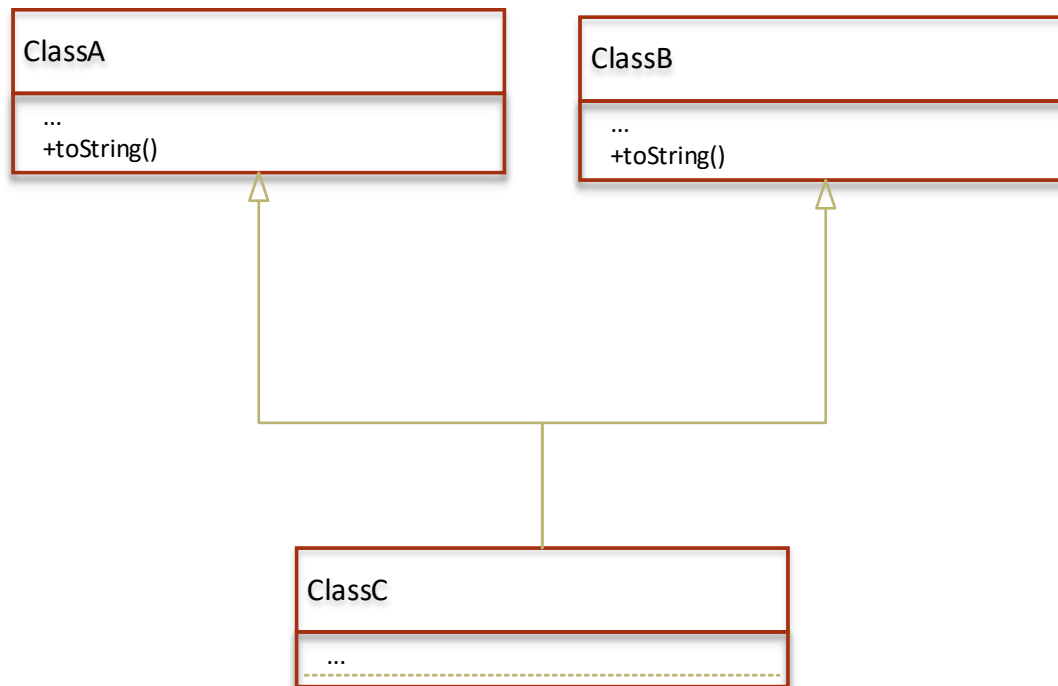


1. NOTE
2. **name:** employees name
3. **staffId:** employee id
4. **calcMonthlyPay:** returns employee's monthly pay
5. **numHours:** number of hours employee worked
6. **annualSal:** employee's annual salary

1. Write the code for **Employee**, **PartTime** and **FullTime** classes. Calculate the monthly pay as follows
 - For **PartTime**: the number of hours worked X rate per hour (assume **hourly rate is R80**)
 - For **FullTime**: $1/12$ of the annual salary
2. Write a class **Tester** to test **Employee**, **PartTime** and **FullTime** classes

8. Multiple inheritance

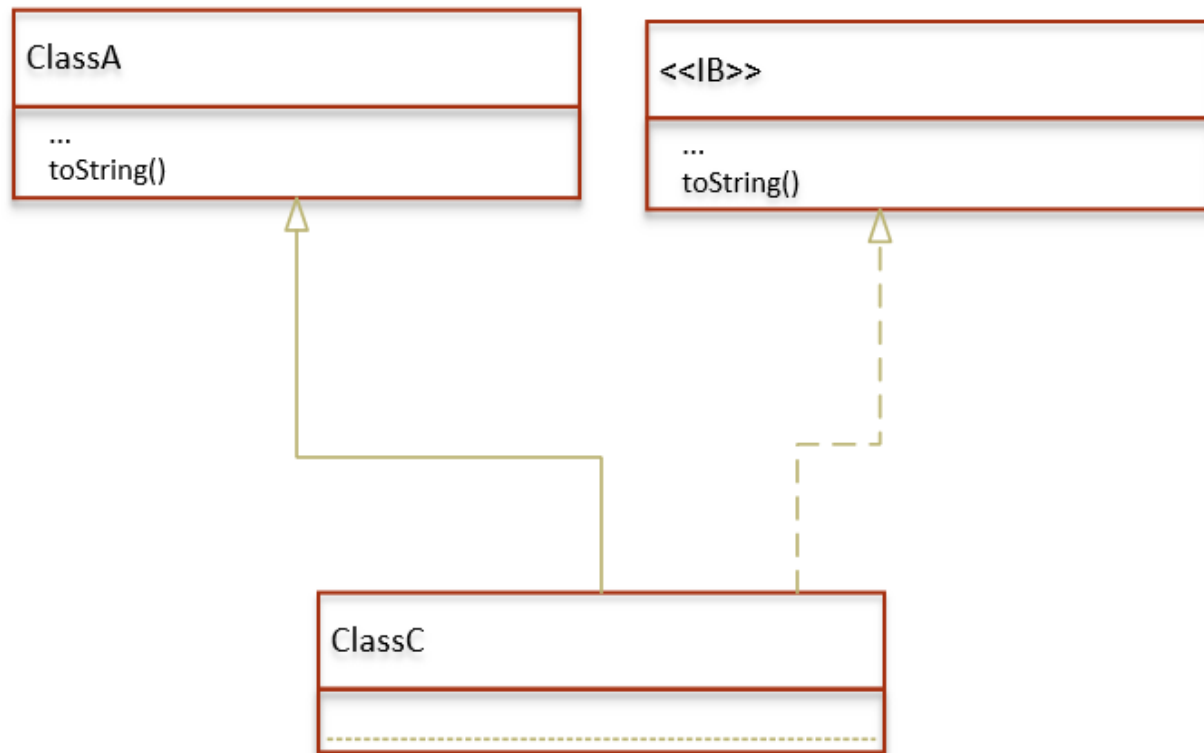
- This occurs when a class inherits two or more super classes
- E.g., class C inherits from Class A and Class B



- There are problems with multiple inheritance
- E.g., in above, **which toString() is invoked at runtime?**

8. Multiple inheritance

- How to resolve multiple inheritance in Java?
 - Inherit from one superclass and convert others to interfaces
 - Class B has been converted to interface
 - We will discuss the rules for converting a class to interface



9. Summary

1. Object, classes and encapsulation
2. Relationships
3. Inheritance and benefits
4. How to code inheritance
5. Method overload and override
6. Polymorphism
7. Concrete vs Abstract classes
8. Why abstract classes
9. How to code abstract classes
10. Multiple inheritance and problems
11. Interfaces to deal with multiple inheritance

10. Questions and Comments

?

Next.....

■ Part 2: Interfaces